

Automatic Structured Quadrilateral Grid Generation via Frame Fields

Marvyn Bailly

December 11, 2025

Abstract

This paper presents an investigation into automatic structured quadrilateral grid generation using frame fields. The research explores the theoretical foundations and practical implementation of frame field-based methods for generating high-quality structured meshes suitable for finite volume methods. We discuss the mathematical framework, implementation details, and demonstrate the effectiveness of the approach through various test cases.

Contents

1	Introduction	3
2	Methodology	3
2.1	Mixed-Integer Solver	3
2.1.1	Problem Formulation	3
2.1.2	Greedy Mixed-Integer Solver	4
2.2	Frame Field Generation	5
2.2.1	Mathematical Formulation	5
2.2.2	Constructing a Smooth Frame Field	6
2.2.3	Finding a Smooth Frame Field	7
2.2.4	Smooth Relaxation Solution	7
2.3	Quadrilateral Mesh Extraction	8
2.3.1	Mesh Cut	9
2.3.2	Compatibility Conditions	9
3	Results	10
3.1	Frame Field Generation Results	10
3.1.1	DAE11 Airfoil Example	10
3.1.2	Rentrant Corner Example	10
4	Discussion	11
5	Conclusion	12
A	Julia Code Listings	14
B	Reference Paper	14

1 Introduction

In Computational Fluid Dynamics (CFD), the generation of high-quality meshes is crucial for accurate simulations which use finite volume methods. Structured quadrilateral grids are particularly advantageous due to their regular connectivity and alignment with flow features. This paper explores the use of frame fields for automatic generation of structured quadrilateral grids, providing a systematic approach to mesh generation that can adapt to complex geometries. In this study, we implement the Mixed-Integer Quadrangulation (MIQ) method in Julia as described by Bommers et al. [1] and evaluate its performance on various geometries.

We aim to generate structured quadrilateral meshes that maintain the following qualities:

- **Individual Cell Quality:** Each quad should be as close to a square as possible.
- **Orientation:** The grid should align with the geometry’s features and flow directions.
- **Alignment:** The grid should resolve sharp features and boundaries accurately.
- **Global Structure:** Points in the mesh known as singularities are required to compute for the Gaussian curvature of a general surface. singularities correspond to irregular vertices in the quad mesh where the number of incident edges differs from four. These should be minimized and strategically placed.

In the following sections, we will detail the methodology used to implement the MIQ algorithm, present the results of our mesh generation experiments, discuss the implications of our findings, and conclude with insights in this area.

2 Methodology

We break the methodology into two main steps: (1) Frame Field Generation and (2) Quadrilateral Mesh Extraction. The core of both steps relies on solving a minimization problem where some of the variables are integers while the others are real-valued. This is known as a Mixed-Integer Program (MIP) which are generally NP-hard to solve. To address this, we implement an approximation algorithm known as a greedy mixed-integer solver as described by Bommers et al. [1].

2.1 Mixed-Integer Solver

2.1.1 Problem Formulation

The minimization of a quadratic energy function $E(x_1, \dots, x_n)$ is an integer problem if $x \in \mathbb{Z}^n$. However we encounter a more general problem where some of the unknown variables $x_1, \dots, x_k$ must be integers while the remaining variables $x_{k+1}, \dots, x_n$ are real-valued.

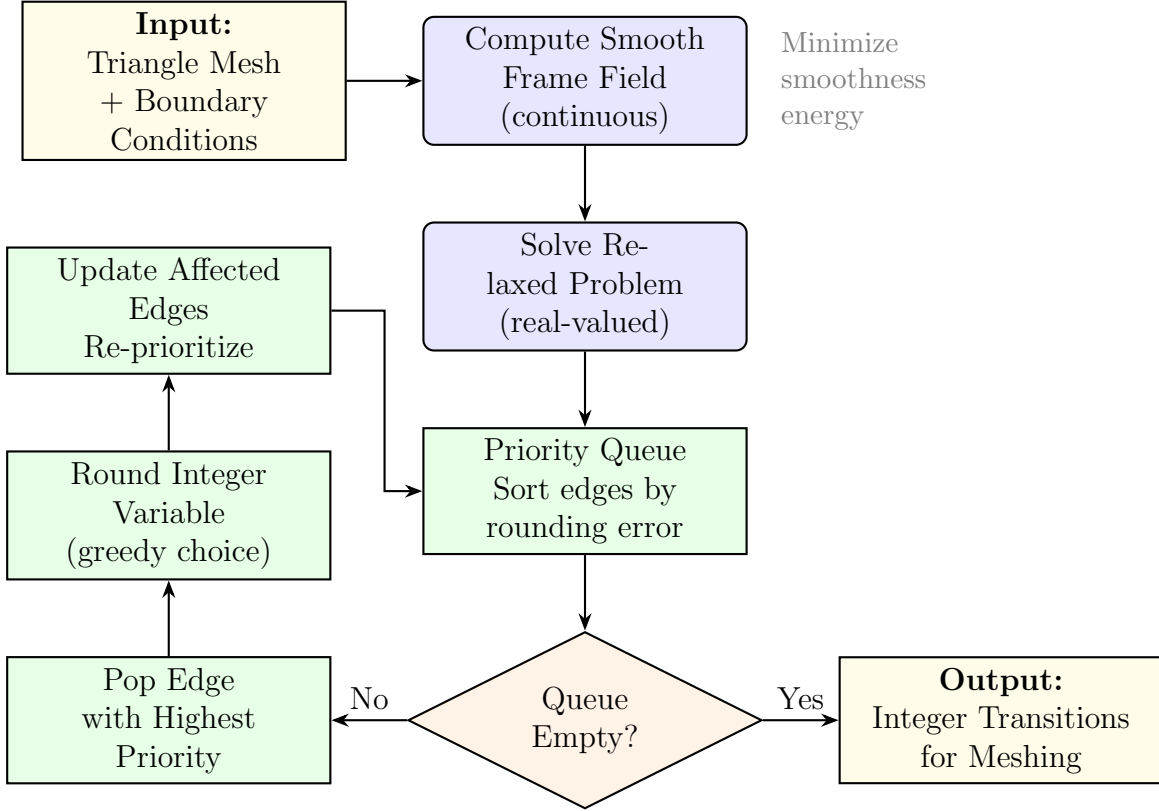


Figure 1: High-level workflow of the greedy mixed-integer solver.

We note that the continuous relaxation of a MIP, obtained by allowing all variables to be real-valued (e.g. relaxing the integer constraints), provides a lower bound on the optimal solution of the original MIP. Thus this provides a reasonable starting point for approximation algorithms to solve MIPs.

2.1.2 Greedy Mixed-Integer Solver

We begin by computing the continuous minimizer of the MIP, which requires the solution of the linear system $\{\partial E / \partial x_i = 0\}_{i=1}^n$. Then the first k variables of the solution are rounded to the nearest integer and treated as constant while resolving the linear system for the remaining real-valued variables. This is known as direct rounding but in this application it often leads to a suboptimal solution. This motivates Bommes et al. [1] to propose a greedy mixed-integer solver.

Rather than rounding the first k real valued variables to integers, the greedy rounding approach rounds one integer variable at a time. This is followed by an immediate update of the continuous part treating the integers as constants.

If we let $\bar{x}^0$ be the continuous solution to the linear system and let x_i , $1 \leq i \leq k$ be the variable which causes the smallest absolute error if we round it to the nearest integer. Then we set x_i to this integer value. We solve the remaining variables $\bar{x}^1$ treating x_i as a constant.

We continue to eliminate in integer variables on each step until we have an integer for all $x_1, \dots, x_k$ as desired by the MIP.

To reduce the computational cost, we use a local Gauss-Seidel update rather than solving the full linear system after each rounding step. That means after x_i is rounded, we add all variables whose Gauss-Seidel update depends on x_i to a priority queue. These elements correspond to the nonzero elements in row A_i . Now on each iteration, we pop the highest priority element from the queue, compute the local residuum

$$r_k = b_k - \sum_{j \neq n} A_{kj} x_j. \quad (1)$$

If $|r_k| \geq \tau$, a small threshold, we update $x_k \leftarrow x_k - r_k/A_{kk}$ and push all variables which depend on x_k back into the priority queue. This continues until the queue is empty. If the method does not converge in a reasonable number of iterations, we fall back to solving the full linear system. Figure 1 illustrates the high-level workflow of this algorithm while Algorithm 1 provides the detailed pseudocode.

2.2 Frame Field Generation

In this section we introduce the mathematical formulation and implementation details for generating a smooth frame field over a given triangular mesh.

2.2.1 Mathematical Formulation

A frame field is a N -symmetry direction field where at each point in the domain, N directions are defined that are symmetric under rotation by $2\pi/N$ around the point of interest. For quadrilateral meshing, we are interested in 4-symmetry frame fields, or cross fields, where $N = 4$. For the rest of the project, we will take $N = 4$. The frame field is represented by an angle-field $\theta : F \mapsto \mathbb{R}$ which assigns an angle θ_f to each face $f \in F$ of the triangular mesh and a period-jump field $p : E \mapsto \mathbb{Z}$ which assigns an integer period-jump p_e to each edge $e \in E$ of the mesh.

The idea is that for each face f , we can define a local frame of reference by treating one of the faces's edge e as the x -axis. Then the four directions of the frame are given by rotating counter-clockwise by $\theta_f + k \cdot \pi/2$ for $k = 0, 1, 2, 3$. This is visualized in Figure 2 a. Since we are aiming to minimize a difference $\theta_i - \theta_j$ for neighboring faces f_i and f_j across the shared edges e_{ij} , we need a way to map one local of reference to another. The period-jump p_{ij} defined on each edge e_{ij} determines the direction in face f_i which corresponds to the direction in the adjacent face f_j . Specifically, the direction in face f_j that aligns with the reference edge is given by $\theta_{f_j} + p_{ij} \cdot \pi/2$. This relationship is illustrated in Figure 2 b.

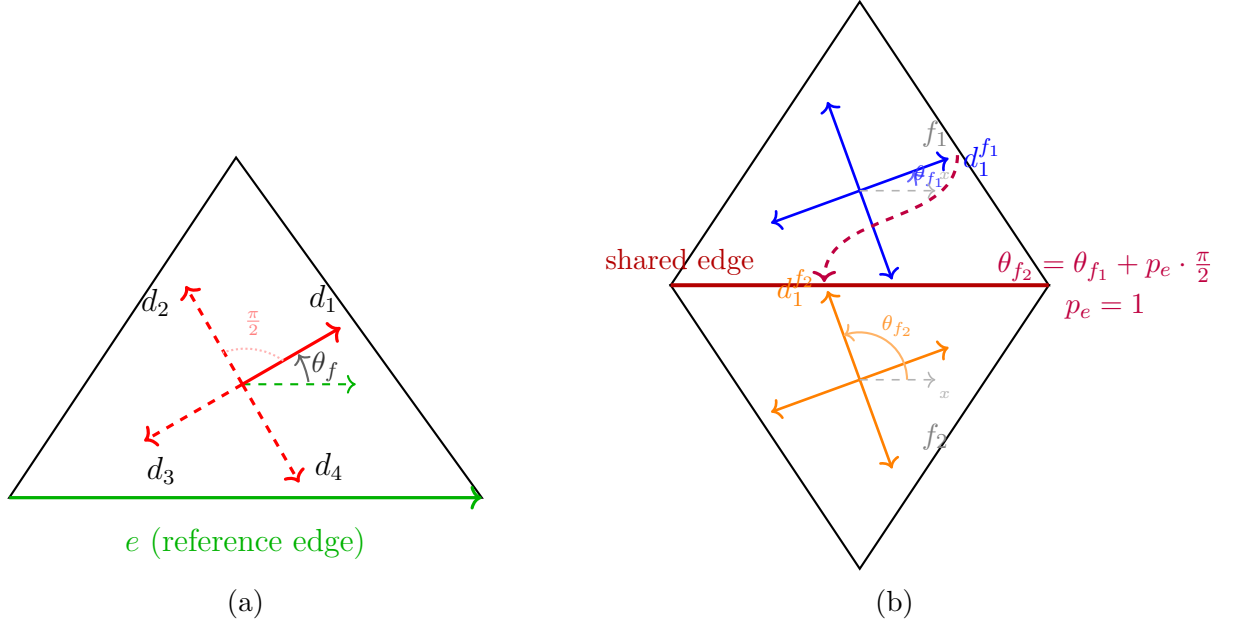


Figure 2: Frame field visualization. (a) The reference edge e is aligned with the x -axis. The frame at face f consists of four directions obtained by rotating the x -axis by angles $\theta_f, \theta_f + \frac{\pi}{2}, \theta_f + \pi, \theta_f + \frac{3\pi}{2}$. (b) The frame field in face f_2 is related to face f_1 by $\theta_{f_2} = \theta_{f_1} + p_e \cdot \frac{\pi}{2}$, where $p_e = 1$ causes a 90° rotation.

2.2.2 Constructing a Smooth Frame Field

The smoothness of a unit frame field can be measured by the angular difference between faces:

$$E_{\text{smooth}} = \sum_{e_{ij} \in E} \left(\theta_i + \kappa_{ij} + \frac{2\pi}{N} p_{ij} - \theta_j \right)^2, \quad (2)$$

where θ_i , and θ_j are the angles of triangle i and j to its respective reference edge, $\kappa_{ij} \in (-\pi, \pi]$ is the angle between the reference edges of triangles i and j , p_{ij} is the integer period-jump across edge e_{ij} , and $N = 4$ for quadrilateral meshing.

The cross field index of a vertex v is defined as:

$$I(v) = I_0(v) + \sum_{e_{ij} \in \text{star}(v)} \frac{p_{ij}}{4}, \quad (3)$$

where $I_0(v)$ is the initial index determined by the geometry of the mesh

$$I_0(v) = \frac{1}{2\pi} \left(A_d(v) + \sum_{e_{ij} \in \text{star}(v)} \kappa_{ij} \right), \quad (4)$$

where $A_d(v)$ is the angle defect at vertex v and $\text{star}(v)$ is the set of edges incident to vertex v . We compute the angle defect as

$$A_d(v) = 2\pi - \sum_{f_k \in \text{star}(v)} \alpha_k, \quad (5)$$

where α_k is the angle at vertex v in face f_k . It can be shown that vertices with non-zero index correspond to singularities in the frame field which will later correspond to irregular vertices in the quadrilateral mesh. The index is ensured to be an multiple of $1/4$ and will determine the valence of the irregular vertex in the quad mesh. An index of $+1/4$ corresponds to a vertex with valence 3 while an index of $-1/4$ corresponds to a vertex with valence 5. In general, we have the relationship

$$\text{valence}(v) = 4 - 4I(v). \quad (6)$$

2.2.3 Finding a Smooth Frame Field

The problem we wish to solve is given a triangular mesh M and a subset of faces $F_c \subset F$ with constrained direction $\theta_i = \hat{\theta}_i$ for $i \in F_c$, we search for a smooth frame field by minimizing the energy given by equation 2. That is, we wish to solve for θ_i and p_{ij} , which minimize E_{smooth} subject to the constraints $\theta_i = \hat{\theta}_i$ for $i \in F_c$. This is a mixed-integer problem since θ_i are real-valued while p_{ij} are integers.

To reduce the problem space, we can take advantage of the invariance under rotation of the frame field. Following the work of Bommers et al. [1], the minimization problem can be made unique by fixing one period jump variable to zero per triangle before the initial continuous solution x^0 . In order to not change the value of the energy, we take care not to fix edges whose dual paths connect two constrained faces, or closes loops. To generate a valid set of edges whose period jumps may be fixed, we construct a Dijkstra tree over the dual graph of the triangular mesh, starting from each of the constrained faces. The algorithm works by each root claiming edges to prevent loops and connected paths between trees. A visualization of this can be seen in the green lines coming the yellow faces in Figure 3. Here the yellow faces represent the constrained faces F_c while the green lines represent the edges whose period jumps have been fixed to zero.

2.2.4 Smooth Relaxation Solution

To find the solution of the continuous relaxation, we take the derivative of the energy function with respect to each variable, leading to a system of linear equations for the real-valued variables θ and now real valued integer constraints for the period jumps p . The resulting equations are as follows:

$$\begin{aligned} \frac{\partial E_{\text{smooth}}}{\partial \theta_k} &= \sum_{e_{kj} \in \text{star}(v)} 2 \left(\theta_k + \kappa_{kj} + \frac{\pi}{2} p_{kj} - \theta_j \right) = 0, \\ \frac{\partial E_{\text{smooth}}}{\partial p_{ij}} &= \pi \left(\theta_i + \kappa_{ij} + \frac{\pi}{2} p_{ij} - \theta_j \right) = 0. \end{aligned} \quad (7)$$

For all variables that are not fixed, we assemble a row in a single matrix that we pass to the greedy mixed-integer solver described earlier. The solver returns the optimal θ and p values which define the smooth frame field over the triangular mesh adhering to the user-defined constraints on F_c . An example of a smooth frame field proceed by my Julia solver around

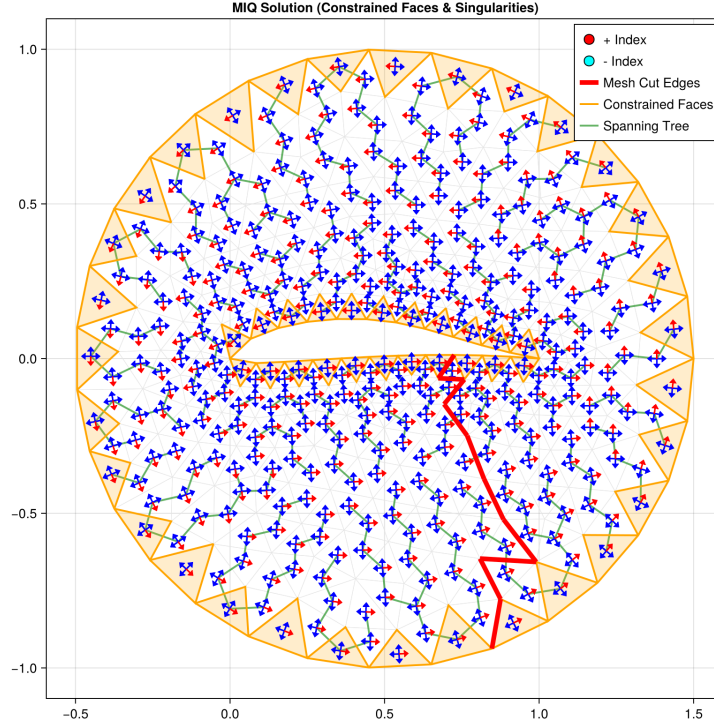


Figure 3: The resulting smooth frame field around a DAE11 airfoil geometry. The frame directions are visualized as blue crosses with the main direction shown in red. The constrained faces are highlighted in yellow. The forest of Dijkstra trees whose edges have fixed period jumps are shown in green. The cut graph used for quadrilateral mesh extraction is shown in red.

a DAE11 airfoil geometry is shown in Figure 3. We note that in this case, there are no singularities in the frame field.

2.3 Quadrilateral Mesh Extraction

To extract a quadrilateral mesh from the computed frame field, we follow the method described by Bommers et al. [1]. We begin by computing a global parameterization from the given mesh to a disk-shaped parameter domain in $\mathbb{R}^2$. To do so, we assign a (u, v) parameter value to each vertex in the mesh such that the parametrization locally aligns to the smooth frame field found in the previous section. That is, the u and v gradients should minimize the following local energy:

$$E_T = \|h\nabla u - \mathbf{u}_T\|^2 + \|h\nabla v - \mathbf{v}_T\|^2, \quad (8)$$

where $\mathbf{u}_T$ and $\mathbf{v}_T$ are two orthogonal directions of the frame field in triangle T and h is a global scaling factor to control the size of the quads in the final mesh. The global orientation

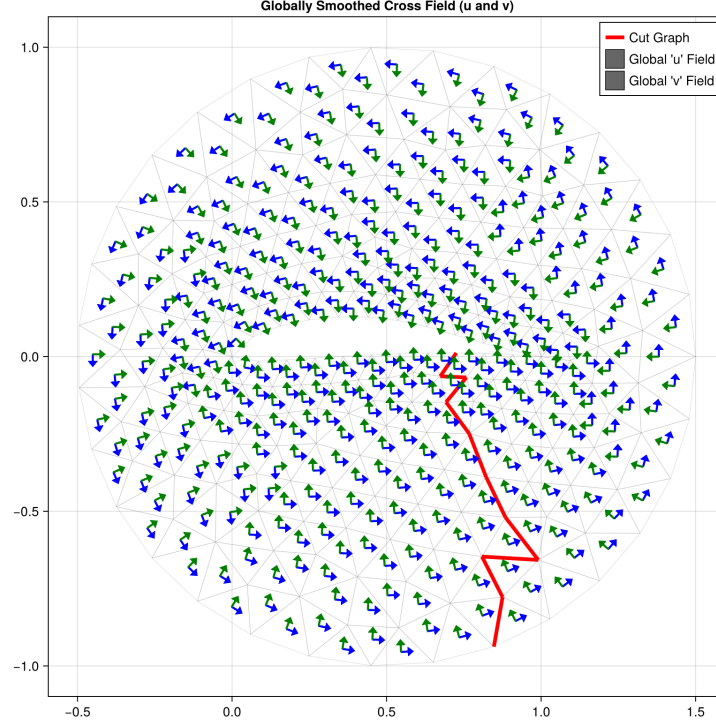


Figure 4: Extracted $\mathbf{u}_T$ and $\mathbf{v}_T$ for the DAE11 airfoil example.

energy is then given by summing over all triangles:

$$E_{\text{orient}} = \int_M E_t dA = \sum_{T \in F} A(T) E_T, \quad (9)$$

where $A(T)$ is the area of triangle T . The extracted $\mathbf{u}_T$ and $\mathbf{v}_T$ for the DAE11 airfoil example are visualized in Figure 4.

2.3.1 Mesh Cut

To compute this minimization, we need to cut the mesh into a disk topology by introducing a cut graph from the boundary to all singularities in the frame field. The cut graph follows a similar approach to the Dijkstra tree used in the frame field generation step. The cut graph for the DAE11 airfoil geometry is visualized in red in Figure 3. Intuitively, the cut graph now allows us to "unwrap" the mesh into a rectangular shape which is topologically equivalent to a disk as desired.

2.3.2 Compatibility Conditions

Since the frame field is symmetric under rotation by $\pi/2$, we can pick which direction θ_i and $\theta_i + \pi/2$ to align the u and v gradients in triangle T_i . We require this by ensuring that the

(u, v) parameters on both sides of a cut edge satisfy

$$(u', v') = \text{Rot}_{\pi/2}^i(u, v) + (j, k),$$

where $\text{Rot}_{\pi/2}^i$ is a rotation by $i \cdot \pi/2$ for $i = 0, 1, 2, 3$ and $(j, k) \in \mathbb{Z}^2$ is an integer translation. This ensures that the parameterization is continuous across the cut edges up to a rotation and translation. To reduce the number of unknowns, the rotation coefficients i can be computed by propagating a global choice of orientation. After cutting, we pick a random face and propagate its orientation in a breadth-first manner to all the neighboring faces across non-cut edges. The rotation across cut edges is then determined by the difference in orientation of the two adjacent faces.

Combining the orientation constraints with the energy minimization, we obtain a mixed-integer problem where we solve for the real-valued (u, v) parameters at each vertex and the integer translations (j, k) at each cut edge. This is solved using the greedy mixed-integer solver described earlier. Finally, we extract the quadrilateral mesh by tracing the integer isolines of u and v over the triangular mesh.

3 Results

Using this methodology, we implement the full pipeline in Julia to generate structured quadrilateral meshes from triangular input meshes. See Appendix A for the full code listings. We present the results for the smooth frame field generation but unfortunately were not able to complete the quadrilateral mesh extraction step in time for this report.

3.1 Frame Field Generation Results

3.1.1 DAE11 Airfoil Example

Looking at the DAE11 airfoil example shown in Figure 3, we see that the generated frame field smoothly curves around the airfoil geometry. We see that there are no singularities in this frame field meaning that the resulting quadrilateral mesh will be fully regular with all vertices having valence 4. To have the frame field align with the airfoil geometry, we constrained the frame directions on the airfoil surface to be tangential to the surface. This is done by setting θ_i for all faces i adjacent to the airfoil boundary to be equal to the angle of the edge aligned with the airfoil surface. Now extracting the $\mathbf{u}_T$ and $\mathbf{v}_T$ directions from the frame field as shown in Figure 4, we see that they also smoothly curve around the airfoil geometry as desired. Finally, we solve for the global parameterization (u, v) over the triangular mesh which aligns with these directions. The resulting parameterization is shown in Figure 5 where we see that the isolines of u and v .

3.1.2 Reentrant Corner Example

The pipeline was run on a corner domain. The solutions are summarized in Figure 6.

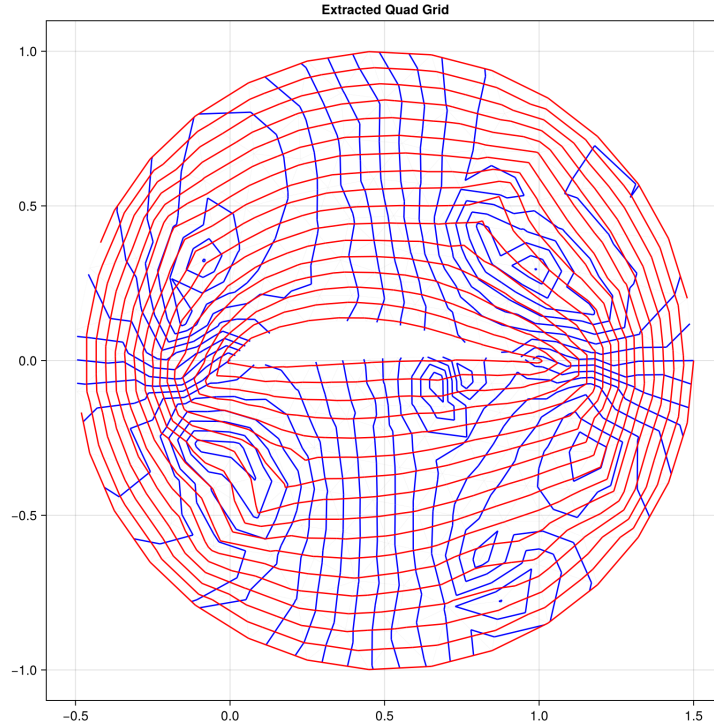


Figure 5: Resulting parameterization (u, v) over the DAE11 airfoil triangular mesh. The isolines of u and v are shown in blue and red respectively.

4 Discussion

Looking at the resulting quadrilateral mesh extracted from the parameterization for the DAE11 airfoil example shown in Figure 5, we see that the mesh has significant distortion in four areas above and below the leading and trailing edge. We see the isolines swirl around a point. I think these are the result of singularities in the frame field that were not properly captured during the computation of the index at each vertex. I'm not confident in my computation of equations (3) to (5) for a general triangulation and I think this is where the issue lies. Once the singularities are properly captured, the algorithm will cut the mesh accordingly and prevent these swirling isolines. Looking at the corner domain example shown in Figure 6, we see similar swirling isolines issues further indicating that the singularities were not properly calculated.

While the greedy mixed-integer solver performed well for the frame field generation step, I found that it struggled to converge for the parameterization step. This is due to our miscalculation of the singularities and the lack of penalization terms in the energy function shown in Equation (9). Bommies et al. [1] introduce additional penalization terms to the energy function to penalize large integer translations across cut edges and to penalize deviation from a uniform scaling factor h . These terms help the solver converge to a better solution. Implementing these terms should be straightforward and is left for future work.

5 Conclusion

In this paper, we have presented a methodology for automatic structured quadrilateral grid generation using frame fields. By implementing the Mixed-Integer Quadrangulation (MIQ) method in Julia, we have not quite demonstrated the ability to automatically generate structured quadrilateral meshes that align with geometric features, we hope the motivation and application is evident. The method also provides an approach to automatic block decomposition for multi-block structured grid generation where each block boundary can taken along the cut edges. Future work will focus on improving the quality of the generated meshes and the robustness of the algorithm for complex geometries. Once I have a working 2D generator, I plan to extend this work to a three-dimensional hexahedral mesh generation using 3D frame fields to provide a complete solution for automatic structured mesh generation in CFD applications. Overall I really enjoyed reading and understanding the paper by Bommès et al. [1] and implementing the algorithm in Julia. The authors made it a wonderful learning experience and introduction to frame field driven grid generation.

References

- [1] David Bommès, Henrik Zimmer, and Leif Kobbelt. Mixed-integer quadrangulation. *ACM SIGGRAPH 2009 papers*, pages 1–10, 2009.

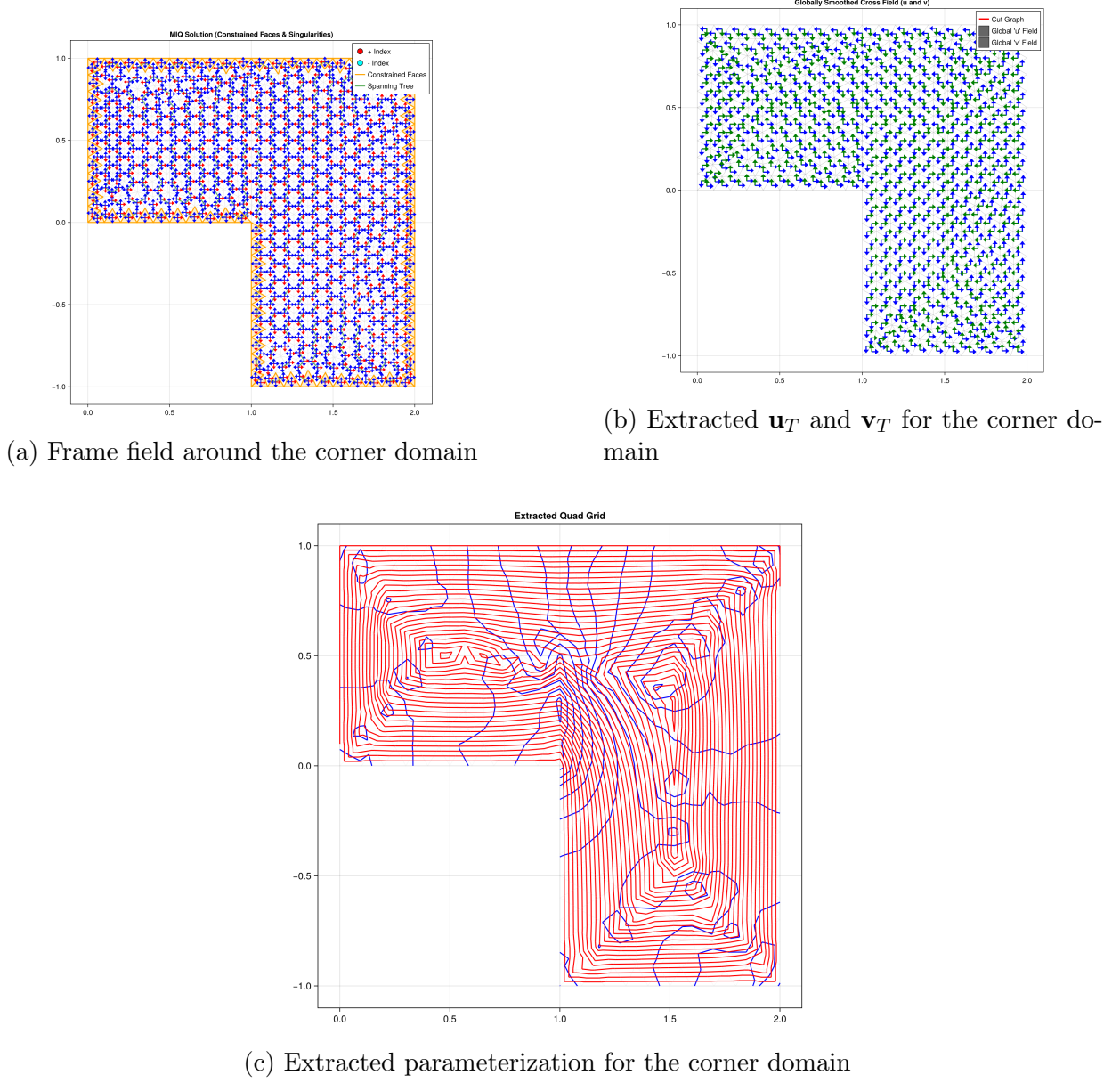


Figure 6: Results for the corner domain example. (a) The resulting smooth frame field around the corner geometry. (b) Extracted $\mathbf{u}_T$ and $\mathbf{v}_T$ for the corner domain. (c) Resulting parameterization (u, v) over the corner triangular mesh. The isolines of u and v are shown in blue and red respectively.

A Julia Code Listings

All code used in this project is available at the following GitHub repository: <https://github.com/MarvynBailly/Structured-Grid-Generation-via-Frame-Fields>.

Algorithm 1: Greedy Mixed-Integer Solver

Input: System matrix A , right-hand side b , number of integer variables k , threshold τ

Output: Solution vector $\vec{x}$ with integer values for $x_1, \dots, x_k$

```
// Solve continuous relaxation
Solve linear system  $A\vec{x}^0 = b$  for all variables;
rounded  $\leftarrow \emptyset$ ;

// Round integer variables greedily
for  $i = 1$  to  $k$  do
    // Find variable with minimum rounding error
     $j \leftarrow \arg \min_{j \in \{1, \dots, k\} \setminus \text{rounded}} |x_j^{i-1} - \text{round}(x_j^{i-1})|$ ;
    // Round and fix variable
     $x_j \leftarrow \text{round}(x_j^{i-1})$ ;
    rounded  $\leftarrow \text{rounded} \cup \{j\}$ ;

    // Update affected continuous variables using Gauss-Seidel
    // Q contains indices  $m$  where  $A_{mj} \neq 0$  (row  $j$  has nonzero entry)
     $Q \leftarrow \{m : A_{mj} \neq 0\}$ ;
    while ( $Q$  is not empty) and (iteration count  $< \text{max\_iterations}$ ) do
         $m \leftarrow Q.\text{pop}()$ ;
        // Compute local residual
         $r_m \leftarrow b_m - \sum_n A_{mn}x_n$ ;
        if  $|r_m| \geq \tau$  then
            // Update variable
             $x_m \leftarrow x_m - r_m/A_{mm}$ ;
            // Add dependent variables to queue
            for each  $n$  where  $A_{nm} \neq 0$  do
                 $Q.\text{push}(n)$ ;
    end while
end for

return  $\vec{x}$ ;
```

B Reference Paper

Include the PDF file (place `bommes2009.pdf` in the same directory)



Mixed-Integer Quadrangulation

David Bommes Henrik Zimmer Leif Kobbelt

RWTH Aachen University

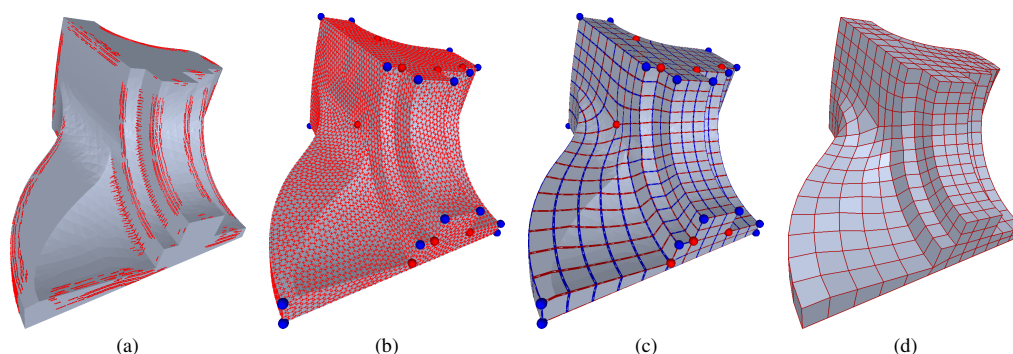


Figure 1: Quadrangulation example: (a) A sparse set of conservatively estimated orientation and/or alignment constraints is selected on the input mesh by some simple heuristic or by the user. (b) In a global optimization procedure a cross field is generated on the mesh which interpolates the given constraints and is as smooth as possible elsewhere. The optimization includes the automatic generation and placement of singularities. (c) A globally smooth parametrization is computed on the surface whose iso-parameter lines follow the cross field directions and singularities lie at integer locations. (d) Finally, a consistent, feature aligned quadmesh can be extracted.

Abstract

We present a novel method for quadrangulating a given triangle mesh. After constructing an as smooth as possible symmetric cross field satisfying a sparse set of directional constraints (to capture the geometric structure of the surface), the mesh is cut open in order to enable a low distortion unfolding. Then a seamless globally smooth parametrization is computed whose iso-parameter lines follow the cross field directions. In contrast to previous methods, sparsely distributed directional constraints are sufficient to automatically determine the appropriate number, type and position of singularities in the quadrangulation. Both steps of the algorithm (cross field and parametrization) can be formulated as a mixed-integer problem which we solve very efficiently by an adaptive greedy solver. We show several complex examples where high quality quad meshes are generated in a fully automatic manner.

CR Categories: I.3.5 [Computational Geometry and Object Modeling]: Hierarchy and geometric transformations

Keywords: remeshing, quadrangulation, parametrization, direction field, singularities, mixed-integer

ACM Reference Format
Bommes, D., Zimmer, H., Kobbelt, L., 2009. Mixed-Integer Quadrangulation. *ACM Trans. Graph.* 28, 3, Article 77 (August 2009), 10 pages. DOI = 10.1145/1531326.1531383
<http://doi.acm.org/10.1145/1531326.1531383>

Copyright Notice
Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or direct commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from Publications Dept., ACM, Inc., 2 Penn Plaza, Suite 701, New York, NY 10121-0701, fax +1 (212) 869-0481, or permissions@acm.org.
© 2009 ACM 0730-0301/2009/03-ART77 \$10.00 DOI 10.1145/1531326.1531383
<http://doi.acm.org/10.1145/1531326.1531383>

1 Introduction

The problem of generating high quality quad meshes from unstructured triangle meshes has received a lot of attention recently. The reason for this interest is that quad meshing converts raw geometric data into a higher representation which effectively supports sophisticated operations like texturing and shape modification. The difficulties in quad meshing arise from the fact that the quality criteria are diverse and their optimization often requires the consideration of global dependencies. The most common quality aspects are:

1. **Individual Element Quality:** Each quad should be close to a rectangle or square, i.e. the four corner points should be coplanar, opposite edges should have equal length and the four interior angles should be 90 degrees.
2. **Orientation:** Away from flat or umbilic points on the surface, mesh edges should be orthogonal to the principal curvature directions such that the dihedral angle across edges captures these curvatures in a natural way.
3. **Alignment:** Sharp features of the surface should be explicitly represented by a sequence of mesh edges in order to minimize the Hausdorff-distance between triangulation and quadrangulation and to prevent normal noise.
4. **Global Structure:** Singularities, i.e. vertices with valence $\neq 4$, are necessary to compensate for the Gaussian curvature. Their number and position must be chosen carefully to capture the global geometric structure since otherwise the element quality and orientation is heavily affected.
5. **Semantics:** In some applications additional requirements emerge from the intended usage of the 3D model and cannot be derived from the geometry alone. In finite element simulation of deformation processes, e.g., the optimal mesh depends

ACM Transactions on Graphics, Vol. 28, No. 3, Article 77, Publication date: August 2009.

77:2 • D. Bommes et al.

on the rest geometry as well as on the external forces and constraints.

A quadrangulation algorithm should optimize the output simultaneously with respect to all of these criteria. However, while element quality, orientation and alignment are rather simple, the global structure is much more difficult to handle. Consequently, we focus on automatically finding good singularity positions which optimize the global structure of the quadrangulation. Although the overall method is designed to run fully automatic, the user can still manually override some decisions, e.g. by manually shifting a singularity wherever it is necessary to take semantical side-conditions into account (see criterion 5).

Many recent methods use smoothed (discrete) principal curvature directions to guide the quad meshing. The problem with these approaches is that the final singularity positions are effectively determined by the local smoothing operator applied to the initial curvature estimates. Especially in flat or umbilic regions where the initial directions have a random orientation, clusters of singularities may occur. Another problem is oversmoothing which may destroy the original orientation information in feature regions.

To overcome these problems we propose to select only the most relevant and dominant directions as depicted in Figure 1 (a), which can be detected, e.g., by conservative thresholding of some anisotropy measure or by manual selection. Starting with these sparsely distributed direction constraints we then search for the smoothest interpolating cross field. The singularities in this interpolating cross field are mostly due to the surface metric deviating from a planar configuration and not caused by incompatible constraints.

In the second phase of our algorithm the smooth cross field is used as input for a global parametrization method. We cut the mesh open such that we create a surface patch with a disk-like topology where all cross field singularities lie at the boundary. Subsequently, we can compute two piecewise linear scalar fields u and v whose gradients follow the given cross field. Finally, a consistent quadrangulation can be extracted since by construction the parametrization is compatible at the cuts and all singularities are mapped to integer positions along the boundary of the parameter domain.

In both steps of the algorithm the task can be formulated in terms of a mixed-integer problem. These are linear problems where a subset of the variables is continuous ($\in \mathbb{R}$) and the others are discrete ($\in \mathbb{Z}$). In Section 2 we therefore present a greedy solver for this class of problem.

1.1 Related Work

A lot of effort has been spent in the last years to compute high quality quadrangulations. Since there are several nice surveys [Alliez et al. 2005; Hormann et al. 2007], we will discuss here only the most related works. Generally, there are two classes of approaches, namely explicit quadrangulations and parametrization based techniques. Examples of explicit approaches are [Alliez et al. 2003; Marinov and Kobbelt 2004] which trace curves along the principal curvature directions or [Lai et al. 2008] which iteratively transforms a triangular mesh into a quad-dominant mesh. For all such methods it is difficult to obtain coarse meshes consisting of quads only. Most structure aligned parametrization techniques are guided by vector or cross fields usually arising from estimated principal curvature directions [Cohen-Steiner and Morvan 2003] or a manual design process [Zhang et al. 2006; Fisher et al. 2007]. Especially cross fields are promising since they can capture singularities of fractional index which naturally arise in quadrangulations.

Cross fields can be seen as four coupled vector fields. Consequently, smoothing algorithms must be able to handle the discretely switching vector assignments, which can be achieved by a non-linear angle formulation like in [Hertzmann and Zorin 2000]. However, such methods often get stuck in local minima and the result strongly depends on the initial solution.

Recently Ray et al. proposed a formalism to handle N-symmetry direction fields [Ray et al. 2008b] in a linear manner, enabling the computation of globally smooth solutions. However, all singularity positions must be prescribed by the user. In contrast we search for singularity positions which enable the smoothest cross field for a set of sparse directional constraints. Besides the automatic singularity placement, another contribution of our approach is the smooth handling of multiple directional constraints, not possible in [Ray et al. 2008b]. Their user-given hard angle constraints already fix the smoothness between multiple constraints, and do not exploit that the orientation of cross fields is invariant w.r.t. rotations by multiples of 90 degrees and that there might be rotations leading to a smoother cross field.

In the case of highly detailed geometry a smooth cross field naturally requires lots of singularities which can be prevented by the smoothing algorithm of Ray et al. [Ray et al. 2008a]. In contrast to their approach, we interpret the cross field as an infinitely fine quadrangulation and compute all necessary singularities. The merging of singularities is later controlled by the parametrization which can perform singularity cancellations w.r.t. the given target edge length.

Structure aware Parametrization techniques can be divided into two classes, namely high-level methods where the quad orientation is controlled by a rough patch layout and low-level methods where a desired orientation is given per triangle. Dong et al. used the Morse-Smale complex of Laplacian eigenfunctions [Dong et al. 2006] to derive high-level patch layouts. Their method was extended in [Huang et al. 2008] to enable the control over singularity positions, size, orientation and feature alignment. However, computing coarse high-quality results is still involved and requires an experienced user. Tong et al. used high-level user-designed singularity graphs to enrich the space of harmonic one-forms and compute globally smooth parametrizations [Tong et al. 2006]. Once a suitable singularity graph is provided by the user, these harmonic parametrizations produce nice quadrangulations. By allowing affine transition functions and optimizing the charts, [Bommes et al. 2009] improved the distortion of the parametrization.

Ray et al. proposed a fully automatic non-linear parametrization technique which is guided by a low-level vector field and assumes a single chart for each triangle [Ray et al. 2006]. Kälberer et al. developed a linear algorithm by mapping a cross field to a single vector field on a branched covering [Kälberer et al. 2007]. As in our parametrization, a triangle based energy is optimized and the singularity positions are completely defined by the input cross field. Their intermediate parametrization, i.e. the integral of the hodge decomposed vector field which is incompatible at the cuts, is exactly the continuous solution of our mixed-integer formulation. However, ensuring compatibility at the cuts is done in a different way. Instead of rounding the coefficients of the transition functions at once, we apply our proposed greedy strategy which improves the resulting quality. A more detailed comparison will be given later in Section 6.

Automatic cone singularities In the setting of conformal parametrizations, cone singularities as introduced by Kharevych et al. [Kharevych et al. 2006] can be placed in a greedy manner at the local extrema of discrete conformal scaling factors [Ben-Chen et al. 2008]. These positions can be further improved by a non-

linear Gauss-Seidel solver [Springborn et al. 2008]. Both methods are designed to compute conformal parametrizations with a small number of cone singularities and lower distortion. Unfortunately, even when restricting to cross field cone singularities, the resulting positions are often not sufficient for structure aligned parametrizations where singularities are additionally induced by the desired orientations, e.g. the leftmost red singularity lying on the flat part of the fandisk in Figure 1. Furthermore, supporting orientational constraints isn't straight forward in this formulation.

1.2 Contributions

We propose an adaptive greedy solver for mixed integer problems which increases the computation time compared to a continuous linear system solver only moderately. This is achieved by iterative rounding combined with local Gauss-Seidel updates in order to reduce the local residui.

We formulate the quadrangulation problem as a two-step process, cross field generation and global parametrization, which both reduce to a mixed-integer problem.

Our cross field generator is able to take sparsely scattered as well as densely distributed orientation constraints into account. By smoothly interpolating between the constraints the system can automatically place singularities at geometrically meaningful locations.

Our new globally smooth parametrization technique allows us to generate seamless quad meshes while satisfying various constraints like orientation, alignment, and integer singularity locations. An optional anisotropic stretch metric allows us to trade squareness of the quads for improved feature alignment.

2 A Greedy Mixed-Integer Solver

The minimization of a quadratic energy $E(x_1, \dots, x_n)$ is called an *integer problem* if $\mathbf{x} \in \mathbb{Z}^n$. In this paper we encounter more general problems where some of the unknown variables $x_1 \dots x_k$ are integers and the others $x_{k+1} \dots x_n$ are real numbers. Integer and mixed-integer problems are usually very hard to solve exactly, see [Floudas 1995; Gorry et al. 1970] for more details. Hence, a common way to find an approximate solution is to first compute the continuous minimizer, which simply requires the solution of the linear system $\{\partial E / \partial x_i = 0 \mid i = 1 \dots n\}$. Then the first k variables of the solution vector are rounded to the nearest integer and a new minimizer is computed by assuming these rounded values $x_1 \dots x_k$ to be constant.

While this *direct rounding* is a common practice, we observed that, depending on the number of integer variables and on their mutual dependencies, the obtained solution can deviate significantly from the true solution. Hence, we propose an alternative approach which we call *greedy rounding*. The idea is to round the integer variables one at a time, followed by an immediate update of the continuous part of the solution.

Let $\mathbf{x}^0$ be the continuous solution to the linear system and let x_i , $i \leq k$ be the variable which causes the smallest absolute error if we round it to the nearest integer. Then we can set x_i to this integer value and update the linear system by assuming x_i as constant. We solve again for the remaining variables $\mathbf{x}^1$ and continue to eliminate in each step that variable which causes the least round-off error until all variables $x_1 \dots x_k$ have an integer value.

The motivation for this approach is based on the assumption that small round-off errors will only have little impact on the final solution and that by recomputing the free variables after every rounding step we will compensate these errors.

The obvious drawback of greedy rounding is that we have to solve k ($=$ number of integer variables) $+ 1$ linear systems, which increases the computation time prohibitively. However, as we said above, small round-off errors only have a small impact on the solution which can be exploited to design an efficient adaptive solver.

2.1 Adaptive Mixed-Integer Solver

The first idea is to use an iterative solver like the Conjugate Gradient or BiCG (for non-symmetric matrices) which after each rounding reuses the previous solution as the initial value. But we can do even better. We observed that for sparse matrices, typically arising in the case of triangle meshes, it is often sufficient to update a small local set of variables. In this context local means that we have a short path in the dependency graph of all variables. Therefore, we start with a local Gauss-Seidel iteration. That means after variable x_i is rounded we push all variables whose Gauss-Seidel update depends on x_i into a queue. These are exactly the nonzero elements of the row A_i . Now in each iteration step we fetch the first element from the queue, say x_k , and recompute the local residuum

$$r_k = b_k - \sum_{j=1}^n A_{kj} x_j$$

If $|r_k|$ is larger than a prescribed tolerance, e.g. 10^{-6} , we update the variable $x_k \rightarrow x_k - r_k / A_{kk}$ and push all variables which depend on x_k onto the queue. The iteration terminates if the queue is empty, i.e. all local residui are within the prescribed tolerance, or a maximum number of iterations is reached.

Algorithm 1 Local Gauss-Seidel

```

1:  $x_i = \text{round}(x_i)$ 
2: push  $\text{nonzero}(A_i)$  into queue
3:  $\text{iter} = 0$ 
4: while ( (not queue empty) and ( $\text{iter} < \text{maxiter}$ ) ) do
5:    $\text{iter} = \text{iter} + 1$ 
6:    $x_k = \text{pop}(\text{queue})$ 
7:    $r_k = b_k - \sum_{j=1}^n A_{kj} x_j$ 
8:   if ( $|r_k| > \text{tolerance}$ ) then
9:      $x_k = x_k - r_k / A_{kk}$ 
10:    push  $\text{nonzero}(A_k)$  into queue
11:   end if
12: end while
```

If the local Gauss-Seidel solver does not converge ($\text{iter} \geq \text{maxiter}$), we first switch to a more global Conjugate Gradient solver and finally, if it is necessary, to a sparse Cholesky solver [Chen et al. 2006]. Such time consuming Sparse-Cholesky computations are only necessary when a variable with large impact is rounded. Our experiments showed that this adaptive solver is more efficient than restricting to pure iterative solvers. More detailed statistics about the solver are given in Section 6.

In the future we plan to publish our implementation of the presented solver which can be applied to arbitrary quadratic mixed-integer problems. For maximum flexibility, we added the handling of linear constraints, by internally eliminating a variable for each independent constraint.

3 Salient Curvature Directions

In the vicinity of flat or umbilic points, the principal curvature directions are ill defined. Consequently, using the principal curvature directions as a dense guiding field for quadrangulation leads to sub-optimal results. Typical artifacts are noisy directions with badly

77:4 • D. Bommes et al.

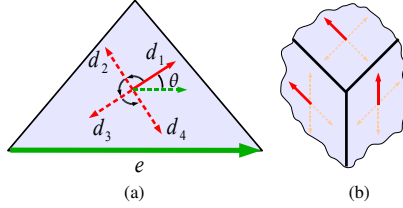


Figure 2: (a) The four cross field directions in a triangle are parametrized by the angle θ w.r.t. a local reference edge e . (b) Depicts a smooth cross field in the vicinity of a cube corner, where the red arrows reflect the corresponding period jumps.

placed singularities or even clusters of unnecessary singularities. Generally, these artifacts cannot be removed by cross field smoothing algorithms, since the configurations often form local minima. Therefore, in contrast to other methods, we aim at finding the smoothest cross field, interpolating only sparse directional constraints that can be found in a reliable manner.

The directions we want to identify are in the spirit of feature lines, as computed in [Hildebrandt et al. 2005]. However in our case a simple heuristic which robustly identifies parabolic regions is sufficient. Since parabolic regions are equipped with a well-defined orientation they are the best candidates to guide a quadrangulation. Parabolic regions can be identified by measuring the relative anisotropy of the principal curvatures

$$\tau = \frac{||\kappa_{max}| - |\kappa_{min}||}{|\kappa_{max}|} \in [0, 1]$$

which is defined to be zero, if κ_{max} is zero.

Computing meaningful curvatures on discrete triangle meshes is involved. A common technique is evaluating the shape operator [Cohen-Steiner and Morvan 2003] of a geodesic disk near a point p . But depending on the radius r we will get different estimates. To achieve a more stable result we compute for each point a set of shape operators S_r with different geodesic radii $r \in [r_0, r_1]$ and select the most promising one with a simple heuristic. A shape operator S_r is said to be valid if all shape operators in the interval $[r - w, r + w]$ have a relative anisotropy larger than a prescribed threshold τ_{min} and a mean curvature larger than K to exclude almost flat regions. For all points which provide a valid shape operator, we add a directional constraint. If there are multiple valid candidates for a single point we choose the one with the most stable direction, i.e. the one with the minimal angle deviation within its interval.

Fortunately all necessary coefficients of this heuristic have an intuitive meaning. Appropriate directions should be stable within a range depending on the target edge length h . Following this observation we choose $w = h/4$. Furthermore in our experiments we chose r_0 to be the average length of all triangle edges, $r_1 = h$, $\tau_{min} = 0.8$ and $K = 0.1/b_s$, where b_s is the radius of a bounding sphere. In general the quadrangulation result is not very sensitive w.r.t. these parameters, since similar cross fields can be generated with a large range of different sparse constraints, generated with slightly different parameters.

ACM Transactions on Graphics, Vol. 28, No. 3, Article 77, Publication date: August 2009.

4 Smooth cross fields

In this section we will use the elegant formalism for N-Symmetry direction fields [Ray et al. 2008b] where a cross field ($N = 4$) on a triangle mesh $M = (V, E, F)$ is defined by an angle-field $\theta : F \mapsto \mathbb{R}$ assigning a real number to each face and a period-jump field $p : E \mapsto \mathbb{Z}$ assigning an integer to each edge. The main idea is to use the angles θ to determine a single unit length vector-field which then extends to a symmetric cross field by applying three rotations of $\frac{\pi}{2}$ as shown in Figure 2 (a). Because a cross consists of four vectors between neighboring triangles it is necessary to identify which vector of the first cross is associated with which vector of the second cross. All these topological issues are handled by the period-jumps, as illustrated in Figure 2 (b) for a smooth cross field near the corner of a cube. In this section we will summarize only the discrete results about cross fields that we need in this paper. For more details see [Ray et al. 2008b].

4.1 Measuring cross field smoothness

After fixing the topology, measuring the smoothness of a cross field reduces to measuring the smoothness of one of the four rotation symmetric vector-fields.

The smoothness of a unit vector-field can be measured as the integrated squared curvature of the direction field. Following [Ray et al. 2008b], on a discrete triangle mesh it turns out to be simply the sum of all squared angle differences between neighboring triangles:

$$E_{smooth} = \sum_{e_{ij} \in E} (\theta_i - \theta_j)^2$$

where θ_i is the angle of triangle i and neighboring angles are represented in a common coordinate frame, which is always possible by flattening both triangles along their common edge. However, for a surface with non-zero Gaussian curvature it is not possible to find a global coordinate frame. Therefore, a local coordinate frame is used for each triangle, where the x axis is identical to the first edge e of the triangle (Figure 2(a)). Thus, by incorporating the coordinate transformations between neighbors we can express the smoothness energy of a cross field:

$$E_{smooth} = \sum_{e_{ij} \in E} \underbrace{(\theta_i + \kappa_{ij} + \frac{\pi}{2} p_{ij} - \theta_j)^2}_{\theta_i \text{ w.r.t. frame } j} \quad (1)$$

where $\kappa_{ij} \in (-\pi, \pi]$ is the angle between both local frames and p_{ij} is the integer valued period jump across edge e_{ij} . The cross field index of a vertex can be computed as

$$I(v_i) = I_0(v_i) + \sum_{e_{ij} \in N(v_i)} \frac{p_{ij}}{4}$$

with the constant integer valued base index

$$I_0(v_i) = \frac{1}{2\pi} \left(A_d(v_i) + \sum_{e_{ij} \in N(v_i)} \kappa_{ij} \right)$$

and $A_d(v_i)$ is the angle defect of vertex v_i . Only singularities of the cross field have a nonzero index which is always a multiple of $\frac{1}{4}$ [Ray et al. 2008b], e.g. $\frac{1}{4}$ and $-\frac{1}{4}$ for quadrangulation configurations corresponding to valence 3 and 5 respectively.

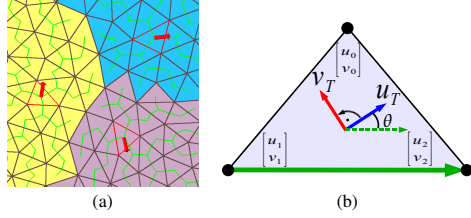


Figure 3: (a) The three constrained faces (red) are the roots of dual spanning trees (green) covering the respective Voronoi cells. Each cell contains only one constraint and along all branches of the tree zero period jumps can be propagated without changing the total smoothness energy. (b) With the angle θ w.r.t. the local reference direction (green) the cross field directions $\mathbf{u}_T, \mathbf{v}_T$ can be extracted and used for the parametrization. In the computation two linear scalar functions (u, v) are sought whose gradients are oriented consistently with the cross field directions.

4.2 Finding a smooth, interpolating cross field

Equipped with these basic definitions we are ready to formulate the optimization problem. Given a mesh M and a subset of faces $F_c \subset F$ with constrained directions $\theta_i = \hat{\theta}_i$, we search for the smoothest interpolating cross field, i.e. we want to minimize (1). Accordingly we have to find an integer p_{ij} per edge and a real valued angle θ_i per face.

Reducing the Search Space: Up to here there is a whole space of equivalent minimizers to the energy (1). To understand this, assume we have already computed a minimizer which for one triangle provides the angle θ_0 and the three period jumps p_{01}, p_{02} and p_{03} . If we now rotate the vector by a multiple of $\frac{\pi}{2}$, i.e. set $\hat{\theta}_0 = \theta_0 + k \cdot \frac{\pi}{2}$ and compensate this change by updating the affected period jumps to $\hat{p}_{0i} = p_{0i} - k$, the smoothness energy is unchanged. We can repeat this procedure for all free triangles $f \in F \setminus F_c$. Consequently the solution can be made unique by fixing one period jump per free triangle to an arbitrary value, e.g. zero, without changing the energy of the minimizer. Care should be taken not to fix edges whose dual path connects two constrained faces, as done in [Ray et al. 2008b], or closes loops because in these cases the cross field curvature along this path would be fixed to an arbitrary value and is not the intended result of the minimizer.

A valid set of edges, whose period jumps are allowed to be set to zero, can be found by constructing a forest of Dijkstra trees of the dual mesh as shown in Figure 3. Each constrained face in F_c is the root of a separate tree such that no tree connects constrained faces. The number of fixed edges is exactly $|F \setminus F_c|$ since starting from the constrained faces each other face of the mesh is conquered by adding a single edge. Notice that no dual loop can be closed by a tree structure, such that we end up with a valid set of edges which can be fixed to zero period jumps without changing the energy of the minimizer.

Obviously there are many other valid sets of edges which could be fixed. The reason why we use trees living in the discrete Voronoi cells of the corresponding constrained faces is that this choice minimizes the length of a path to its corresponding constraint and so improves the accuracy of the greedy mixed-integer solver.

Additionally to the period jumps on tree edges each period jump between two adjacent constrained faces f_i and f_j can be fixed to

Mixed-Integer Quadrangulation • 77:5

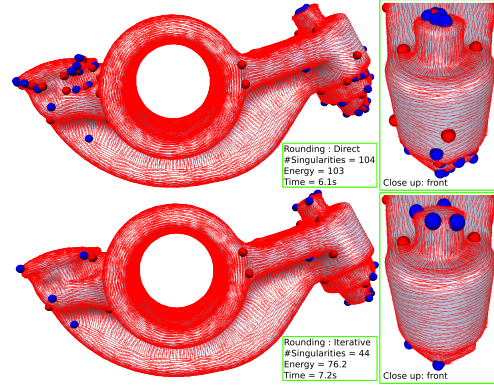


Figure 4: Greedy rounding yields a smaller smoothness energy and fewer singularities (bottom), whereas the direct rounding produces unnecessary singularities and a higher energy (top). Note that these are the singularities and the field as they emerge from the solver, no singularity optimization has been carried out.

$p_{ij} = \text{round}(2/\pi(\hat{\theta}_j - \hat{\theta}_i - \kappa_{ij}))$, since p_{ij} is only part of a single quadratic term in (1), which is independent from other variables.

In summary we end up with a mixed-integer problem consisting of $|F \setminus F_c| \approx 2|V|$ real valued variables θ_i and $|E| - |F \setminus F_c| \approx |V|$ integer valued variables p_{ij} .

Mixed-Integer Formulation: To apply the greedy mixed-integer solver from Section 2 it is sufficient to assemble the system of linear equations by setting the gradient of the energy (1) to zero:

$$\frac{\partial E_{\text{smooth}}}{\partial \theta_k} = \sum_{e_{kj} \in N(f_i)} 2(\theta_k + \kappa_{kj} + \frac{\pi}{2} p_{kj} - \theta_j) \stackrel{!}{=} 0 \quad (2)$$

$$\frac{\partial E_{\text{smooth}}}{\partial p_{ij}} = \pi(\theta_i + \kappa_{ij} + \frac{\pi}{2} p_{ij} - \theta_j) \stackrel{!}{=} 0 \quad (3)$$

Notice that the values on edges are antisymmetric, i.e. $p_{ij} = -p_{ji}$ and $\kappa_{ij} = -\kappa_{ji}$, which can lead to sign changes in equations (2) and (3). For all variables which are not fixed, we set up a row and assemble all of them into a single matrix. After applying our greedy mixed-integer solver, the result is a smooth cross field where the integer valued period jumps define type and position of all singularities. Figure 4 compares the result of our greedy solver with that of a direct rounding, where red and blue spheres represent singularities with negative and positive index respectively.

In practice we observed that some singularity positions, especially those in flat regions, can sometimes be improved by a local search algorithm, as described in the next section.

Local Search Singularity Optimization: In a postprocess we optionally check for each singularity, if the energy can be decreased by moving it to a neighboring vertex. Moving a singularity along an edge e_{ij} means changing the corresponding period jump p_{ij} . Notice, that by this operation only the right-hand-side of the linear system is changed. Consequently we can precalculate the sparse Cholesky factorization of this matrix once and then compute solutions for different right-hand-sides efficiently [Botsch et al. 2005].

77:6 • D. Bommes et al.

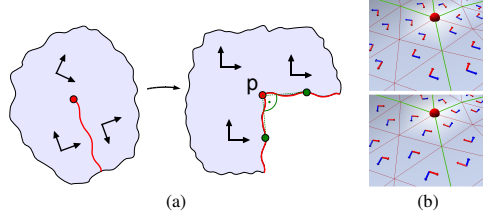


Figure 5: (a) By placing a cut to a cone singularity $\mathbf{p}$ (here of index $\frac{1}{4}$) a distortion free unfolding of the patch is possible. (b) The upper image shows two directions of the cross field. In the lower image the mesh is cut into disk topology along the green edges, such that these directions can be consistently oriented on each side of the cut.

5 Global Parametrization

We now compute a global parametrization, i.e., a map from the given mesh $\mathcal{M}$ to some disk-shaped parameter domain $\Omega \in \mathbb{R}^2$. Since the parametrization should be piecewise linear, it is sufficient to assign a (u, v) parameter value to each vertex — more precisely to each triangle corner — in the mesh.

The parametrization should be locally oriented according to the optimized cross field from Section 4 which implies that the gradients of the piecewise linear scalar fields u and v defined on the mesh $\mathcal{M}$ should minimize the local orientation energy

$$E_T = \|h\nabla u - \mathbf{u}_T\|^2 + \|h\nabla v - \mathbf{v}_T\|^2$$

for each triangle T . Here h is a global scaling parameter which controls the edge length of the resulting quad mesh. The vectors $\mathbf{u}_T$ and $\mathbf{v}_T$ are two orthogonal vectors in T corresponding to the cross field directions θ and $\theta + \pi/2$. Since the cross field is defined only up to rotations by $\pi/2$ we will have to specify which of the four possibilities we are picking in each triangle such that the proper compatibility conditions are satisfied across each edge in the mesh.

The global orientation energy is then defined as the integral of E_T over the entire mesh $\mathcal{M}$

$$E_{\text{orient}} = \int_{\mathcal{M}} E_T dA = \sum_{T \in \mathcal{M}} E_T \text{area}(T). \quad (4)$$

The minimizer of this quadratic functional is obtained by solving the sparse linear system which sets all the partial derivatives of E_{orient} to zero.

Cutting the mesh: In order to be able to compute a proper parametrization minimizing E_{orient} we have to cut open the mesh $\mathcal{M}$, such that we obtain a patch that is topologically equivalent to a disk. An additional requirement is that all singular vertices must lie on the cut, i.e. at the boundary of the parameter domain. The reason is that the angle defect of a singularity cannot be represented by an inner vertex of the parametrization as depicted in Figure 5. We compute an appropriate cut graph in two steps.

First we start from a random triangle and grow a topological disk by constructing a dual spanning tree. Thus the primal of all non spanning tree edges is already a cut graph which transforms $\mathcal{M}$ into disk topology. The size of this cut graph can be significantly reduced by iteratively removing all open paths.

In the second step paths connecting each singularity to the cut graph are added. This can be done by successively applying Dijkstra's shortest path algorithm.

ACM Transactions on Graphics, Vol. 28, No. 3, Article 77, Publication date: August 2009.

At the end of the two cutting steps we have a triangle mesh patch where all the singularities are located at the boundary. If a singularity is not a leaf node of the cut graph then it appears several times along the boundary. In order to compute a parametrization we have to find a planar embedding of this boundary polygon as well as all the interior vertices. The location of the mesh vertices in the parameter domain is computed by minimizing E_{orient} , however, there are a number of consistency constraints that have to be taken into account.

Integer location of singularities: By allowing a singularity to be in general position, it would cause an n -sided face instead of a valence- n vertex. Therefore to guarantee a pure quadrangulation, we have to snap all singularities to integer locations in the parameter domain. This means that the overall parametrization task is now a mixed-integer problem which we solve by our mixed-integer greedy solver from Section 2.

Cross boundary compatibility: In order to avoid visible seams across the cut paths on the surface we have to make sure that the quad structure on both sides of a cut edge is compatible. This is guaranteed by allowing only a grid automorphism as a transition function. This requires that the (u, v) parameter values on both sides of a cut edge are related by

$$(u', v') = \text{Rot}_{90}^i(u, v) + (j, k)$$

with integer coefficients (i, j, k) .

The rotation coefficient in the transition functions can easily be computed by propagating a globally consistent orientation in the cross field, as illustrated in Figure 5. Since after the cutting, all interior vertices of the mesh are regular, we can start at a random face and propagate its orientation in a breadth first manner to all the neighboring faces. This will establish a zero-rotation across all inner edges. The rotations Rot_{90}^i across the cut edges can be found by simply comparing the orientations in neighboring faces.

After fixing the rotations, the cross boundary compatibility conditions can be incorporated into the optimization scheme as linear constraints. Therefore for each cut edge $\mathbf{e} = \overline{\mathbf{p}\mathbf{q}}$ we introduce two integer variables $j_{\mathbf{e}}, k_{\mathbf{e}}$ to formulate the four compatibility conditions:

$$\begin{aligned} (u'_p, v'_p) &= \text{Rot}_{90}^{i_{\mathbf{e}}}(u_p, v_p) + (j_{\mathbf{e}}, k_{\mathbf{e}}) \\ (u'_q, v'_q) &= \text{Rot}_{90}^{i_{\mathbf{e}}}(u_q, v_q) + (j_{\mathbf{e}}, k_{\mathbf{e}}) \end{aligned}$$

Hence, in total we add two integer variables and eliminate four continuous variables per cut edge.

Applying our mixed-integer greedy solver to this parametrization task can be understood in an intuitive way. After computing an all-continuous solution, which corresponds to the unconstrained parametrization, we iteratively snap the singularities to integer locations.

5.1 Anisotropic Norm

In practice exact orientation is often more important than exact edge length. The reason is that changing the orientation along a highly curved feature line, the quadrangulation quality will drop off dramatically due to normal noise. The orientation can be improved by less penalizing stretch which is in the direction of the desired iso-lines. This can be achieved by an anisotropic norm

$$\|(u, v)\|_{(\alpha, \beta)}^2 = \alpha u^2 + \beta v^2$$

which penalizes the deviation along the major directions with different weights. Notice, such a diagonal metric is sufficient since we

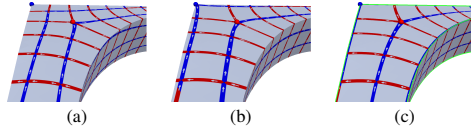


Figure 6: The parametrization in (a) is not aligned to the sharp edges of the object. Using the anisotropic norm the quads are allowed to stretch in order to better align with a given input field as shown in (b). In (c) alignment constraints have been imposed, leading to perfectly preserved features.

use $(\mathbf{u}_T, \mathbf{v}_T)$ as the local coordinate frame in each triangle.

$$E_T = \|\mathbf{h} \nabla \mathbf{u} - \mathbf{u}_T\|_{(\gamma,1)}^2 + \|\mathbf{h} \nabla \mathbf{v} - \mathbf{v}_T\|_{(1,\gamma)}^2$$

with $\gamma \leq 1$. Figure 6 (b) shows an example, where the orientation of the parametrization is improved by using the anisotropic norm.

5.2 Feature Line Alignment

Sharp feature lines of the input mesh should be preserved in the quadrangulation. Given a subset $S \subset E$ of triangle mesh edges, the necessary alignment conditions can be incorporated in a straightforward way. First of all, alignment requires correct orientation. Therefore, while computing the cross field, all edges in S are used as orientational constraints in both adjacent triangles. Additionally to the correct orientation for alignment, a constant integer coordinate along the edge is necessary, which guarantees that this edge is preserved in the quadrangulation. Each alignment condition for an edge $\overline{pq}$ can be formulated independently. If the cross field direction $\mathbf{u}_T$ is already oriented along the alignment edge, we end up with a simple condition for the v parameter values

$$v_p = v_q \in \mathbb{Z}$$

which ensures that $\overline{pq}$ is mapped to an integer valued iso-line. The $u = \text{const}$ case is handled analogously. Consequently, for each alignment edge a single variable can be eliminated and the remaining integer variables can be handled by the greedy mixed-integer solver. Figure 6 (c) shows an example, where all feature edges are aligned.

Notice that for meshes with boundaries we can exploit the presented alignment functionality to guarantee that the boundaries are preserved in the quadrangulation and thus prevent jagged boundary lines (see Figure 9).

5.3 Singularity Relocation

By computing a parameterization with the given target edge length h , new requirements have to be taken into account which cannot be anticipated by the cross field computation, since it is independent from h . Examples are singularities which are too close to each other, a boundary or a given alignment edge. Other aspects are symmetries which are irrelevant for a smooth cross field, but important for a quadrangulation. Therefore, to achieve maximal quality it can be necessary to relocate the singularities w.r.t. the requirements of the parameterization. This can be done with a local search algorithm similar to Section 4.2. Depending on how much time is available we can restrict the search to the best local candidate, i.e. the closest neighbor in the parametrization, or evaluate the quality of all neighbors. In each step it is necessary to recompute the smooth cross field w.r.t. the relocated singularity as well as the parametrization. In the cross field computation the cross field indices are now

Mixed-Integer Quadrangulation • 77:7

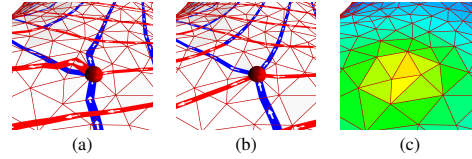


Figure 7: In (a) the minimized orientation energy produces flipped triangles, which can be removed by local stiffening (b). The chosen weighting is shown in (c) and decreases from orange to blue.

prescribed by linear constraints. Movements are performed if the overall quality improves, i.e. the energy (4) decreases.

The obvious drawback of this singularity relocation is its heavy computational cost. Fortunately in all of our examples the initial singularity positions were already sufficient. However, coarsely quadrangulating meshes with fine details will require singularity relocation.

5.4 Local Stiffening

The parametrization is the result of a quadratic energy minimization. Thus despite the global optimum, for a few triangles it might happen that the metric distortion gets very high or even worse, that the orientation of a mapped triangle flips. Figure 7 shows an example where such a problem occurs in the vicinity of a singularity. The idea of local stiffening is to add an adapted triangle weighting $w(T)$ into the energy formulation to penalize high local distortions, yielding:

$$E_{\text{orient}} = \sum_{T \in \mathcal{M}} w(T) E_T \text{area}(T)$$

This weighting, which is initialized to one, can be updated iteratively, as described in the following, until the quality of the parametrization is sufficient.

The metric distortion is characterized by the singular values σ_1 and σ_2 of the Jacobi matrix as described in [Hormann et al. 2007]. Furthermore to penalize flips we evaluate the orientation of a triangle

$$\tau = \text{sign}(\det \begin{bmatrix} u_1 - u_0 & u_2 - u_0 \\ v_1 - v_0 & v_2 - v_0 \end{bmatrix})$$

where (u_i, v_i) are the vertex parameter coordinates in counter-clockwise ordering. We measure the local distortion of each triangle by

$$\lambda = |\tau \frac{\sigma_1}{h} - 1| + |\tau \frac{\sigma_2}{h} - 1|$$

which respects the edge length h . Finally, we update the weight of a triangle by evaluating a uniform Laplacian defined on the dual mesh

$$w(T) \leftarrow w(T) + \min\{c \cdot |\Delta \lambda(T)|, d\}$$

with the proportionality constant c and a maximal allowed update of d , which we chose as $c = 1$ and $d = 5$ in all our examples. Notice that directly using the distortion instead of the Laplacian wouldn't be a good idea. The reason is that the weighting would reflect the *global* stretch distribution, which is necessary for a globally consistent quadrangulation, instead of the desired *local* distortions. Subsequently, we increase the smoothness of the weighting field $w(T)$ by a few uniform smoothing steps, which in general leads to nicer quadrangulations.

77:8 • D. Bommes et al.

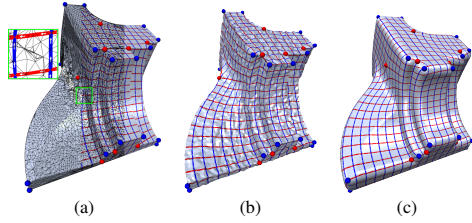


Figure 8: The presented algorithm is robust w.r.t. bad triangles (a) and can produce meaningful singularities in the presence of noise (b) and on smooth offset geometries (c).

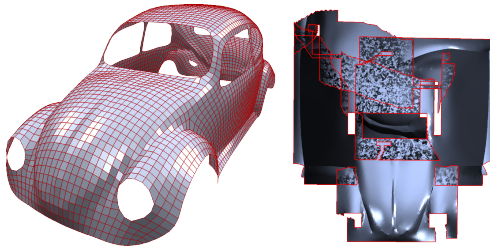


Figure 9: Quadrangulation of the BEETLE model having 11 boundaries. On the right the parametrization is shown. Naturally, due to the occurrence of $-\frac{1}{4}$ singularities, parts of the flattening are overlapping.

6 Comparisons and Results

The backbone of our approach is the mixed-integer solver introduced in Section 2, which is used for the computation of both the smooth cross field and the parametrization. Although it is often necessary to round tens of thousands of variables for the cross field computation, the timings in Table 1 show that this can be done efficiently using the proposed solver.

The example in Figure 4 shows that the greedy rounding leads to a significantly smoother cross field with less singularities compared to the direct rounding approach. All our experiments confirmed this behavior.

A comparison between our approach and QuadCover [Kälberer et al. 2007] is carried out in Figure 10. In both examples the same input cross field and target edge length have been used. The FANDISK comparison clearly shows the benefit of alignment on models with sharp feature edges, while the limitations of direct rounding are especially noticeable on the BOTIJO. On complex objects having many singularities or when remeshing with very coarse target edge length the direct rounding generates many “twists” and non-injectivities in the parametrization, such that the extraction of a hole-free quad mesh is not always possible. However, the combination of greedy rounding and local stiffening allow us to automatically generate consistent, hole-free quadrangulations at almost any resolution and with significantly less “twists”.

The spectral approach [Huang et al. 2008] also produces oriented and aligned parametrizations with few singularities, however the Morse-Smale Complex sometimes fails to capture the detailed structure of the surface. This can lead to an unfavorable stretch

ACM Transactions on Graphics, Vol. 28, No. 3, Article 77, Publication date: August 2009.

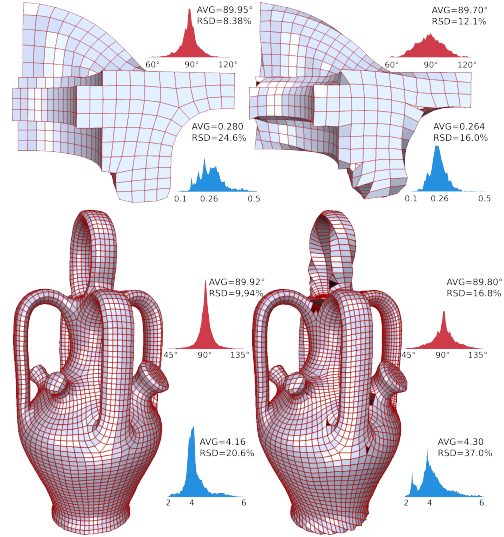


Figure 10: A comparison between the technique described in this paper (left) and the QuadCover approach by [Kälberer et al. 2007] (right) for a sharp object (FANDISK) and a smooth object (BOTIJO). In both comparisons the same target edge length and the same cross field generated by our mixed-integer formulation were used.

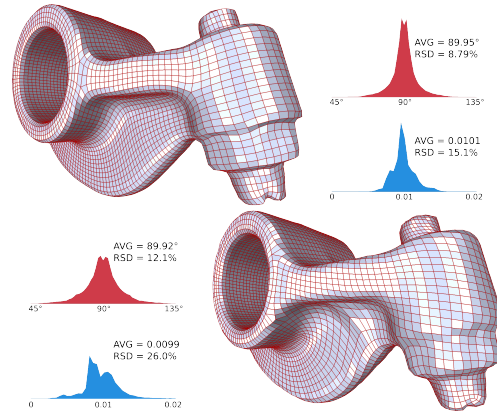


Figure 11: ROCKERARM comparison between the technique described in this paper (top) and the spectral quadrangulation approach by [Huang et al. 2008] (bottom). The upper mesh has 9413 faces and 36 singularities, the lower one has 9400 faces and 26 singularities.

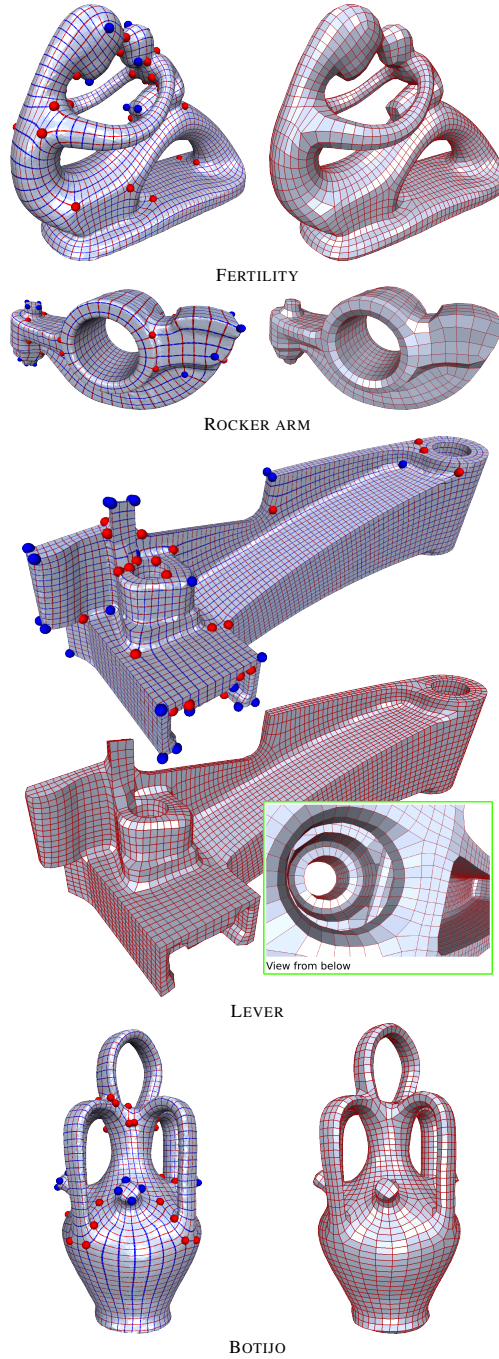


Figure 12: Results of our Mixed-Integer Quadrangulation approach

Mixed-Integer Quadrangulation • 77:9

Model	Solver Statistics										Quadmesh statistics	
	Cross Field					Parametrization					#Sing	#Quad
	Dim	#Int	#IS	#DS	Time	Dim	#Int	Time				
FERTILITY	29342	10621	762	11	6.6s	27954	108	25.7s	48	3357		
ROCKERARM	32843	12064	385	7	7.6s	41552	72	25.5s	36	1127		
LEVER	21113	8254	62	2	3.3s	19331	148	19.9s	83	7880		
BOTIJO	25821	9611	669	7	5.4s	29994	164	44.0s	74	8395		
FANDISK	17820	6447	209	6	2.2s	13776	19	2.4s	30	764		
BEEBLE	46425	15827	562	14	13.3s	34705	82	10.3s	6	3778		

Table 1: Statistics of the Greedy Mixed-Integer Solver used for computing the cross field (Section 4) and the parametrization (Section 5). Dim refers to the initial dimension of the linear system, #Int is the number of integer variables, #IS and #DS is the number of calls to iterative and direct solvers respectively. Time is the total time for the solution. Due to the global nature of the parametrization, the local and iterative search seldom lead to a gain of efficiency and therefore Time refers solely to the direct solver.

of the quads affecting the angle as well as the edge length distribution. A comparison between [Huang et al. 2008] and our approach can be found in Figure 11.

Quadrangulations computed by our technique typically have angle distributions with a sharp peak around 90° and an edge length distribution centered around the target edge length. However, for aligned meshes, like the FANDISK in Figure 10, further peaks, which reflect the unavoidable stretch may occur in the edge length histogram.

The geometrically complex examples shown in Figure 12 underline the ability of our method to compute coarse, oriented quadrangulations with naturally placed singularities.

All examples were computed on a 3.0GHz standard PC, the statistics are shown Table 1. Interestingly the cross field computation is less demanding to compute than the parametrization, even though it requires practically two orders of magnitude more roundings. This effect is due to the locality of the cross field energy (Equation (1)). Rounding a period jump mainly affects a local neighborhood on the mesh and the solution can be efficiently updated by local Gauss-Seidel iterations. Whereas, rounding a corner point in the parametrization domain usually has global impact. Motivated by this observation and the typically low number of integer variables for the parametrization, we restricted the greedy solver to sparse Cholesky updates.

Finally Figure 8 demonstrates the robustness of the mixed-integer quadrangulation approach w.r.t. different (degenerate) representations of a given object. The mesh in Figure 8 (a) contains almost 1000 triangles with vanishing area (the close-up shows a part of the mesh where about 8 triangles are nearly colinear), the model in Figure 8 (b) has been subjected to normal noise with a magnitude of 0.3% of the bounding box diagonal and the right most model (Figure 8 (c)) was offset, yielding a mesh without sharp corners. These fandisks and most of the other triangle meshes used in this work (along with the extracted quad meshes) can be found in the supplementary material of this paper.

7 Conclusion and Future Work

We have presented a complete quadrangulation method which starts with a pre-process that finds reliable orientation constraints. Based on these, possibly sparsely distributed, constraints we compute a smooth cross field on the surface. The global optimization produces a set of singularities that are automatically placed at geometrically meaningful locations. The cross field is used as input for a global

77:10 • D. Bommes et al.

parametrization technique which cuts the surface open into a disk-like patch and then computes a planar embedding which takes orientation, alignment, as well as boundary compatibility constraints into account.

In the future we would like to integrate both parts of the algorithm into a single optimization scheme. Instead of making the parametrization smooth by least squares approximation of a smooth cross field it would be more natural to smooth the parametrization directly. However this would most probably lead to a non-linear optimization.

One limitation of our method is that for coarse quadrangulations of highly complex models with many cross field singularities, the local singularity relocation in the parametrization step is dominating the overall computation time. Here we would like to develop a more global search strategy.

Acknowledgements

This work has been supported by the UMIC Research Centre, RWTH Aachen University. We would like to thank Felix Kälberer and Matthias Nieser for their helpful support, Tamal Dey, Muiyang Zhang, *AIM@SHAPE* and Carlos Hernández (www.tsi.enst.fr/3dmodels) for providing us with datasets, Jan Möbius for the geometry processing framework *OpenFlipper.org* and the reviewers for their competent and helpful comments.

References

- ALLIEZ, P., COHEN-STEINER, D., DEVILLERS, O., LEVY, B., AND DESBRUN, M. 2003. Anisotropic polygonal remeshing. *ACM Trans. Graph.* 22, 3, 485–493.
- ALLIEZ, P., UCELLI, G., GOTSCHMAN, C., AND ATTENE, M. 2005. Recent advances in remeshing of surfaces. Research report, *AIM@SHAPE Network of Excellence*.
- BEN-CHEN, MIRELA, GOTSCHMAN, CRAIG, BUNIN, AND GUY. 2008. Conformal flattening by curvature prescription and metric scaling. *Computer Graphics Forum* 27, 2 (April), 449–458.
- BOMMES, D., VOSSEMER, T., AND KOBBELT, L. 2009. Quadrangular parameterization for reverse engineering. Lecture Notes in Computer Science, to appear.
- BOTSCH, M., BOMMES, D., AND KOBBELT, L. 2005. Efficient linear system solvers for mesh processing. In *IMA Conference on the Mathematics of Surfaces*, Springer, R. R. Martin, H. E. Bez, and M. A. Sabin, Eds., vol. 3604 of *Lecture Notes in Computer Science*, 62–83.
- CHEN, Y., DAVIS, T. A., HAGER, W. W., AND RAJAMANICKAM, S. 2006. Algorithm 8xx: Cholmod, supernodal sparse cholesky factorization and update/downdate. Technical Report TR-2006-005, University of Florida.
- COHEN-STEINER, D., AND MORVAN, J.-M. 2003. Restricted delaunay triangulations and normal cycle. In *SCG '03: Proceedings of the nineteenth annual symposium on Computational geometry*, 312–321.
- DONG, S., BREMER, P.-T., GARLAND, M., PASCUCCI, V., AND HART, J. C. 2006. Spectral surface quadrangulation. In *SIGGRAPH '06: ACM SIGGRAPH 2006 Papers*, 1057–1066.
- FISHER, M., SCHRÖDER, P., DESBRUN, M., AND HOPPE, H. 2007. Design of tangent vector fields. *ACM TOG* 26, 3, 56.
- FLOUDAS, C. A. 1995. *Nonlinear and Mixed-Integer Optimization Fundamentals and Applications*. Hardback.
- GORRY, G., SHAPIRO, J., AND WOLSEY, L. 1970. *Relaxation methods for pure and mixed integer programming problems*. Cambridge, M.I.T., Cambridge.
- HERTZMANN, A., AND ZORIN, D. 2000. Illustrating smooth surfaces. In *SIGGRAPH '00: Proceedings of the 27th annual conference on Computer graphics and interactive techniques*, ACM Press/Addison-Wesley Publishing Co., New York, NY, USA, 517–526.
- HILDEBRANDT, K., POLTHIER, K., AND WARDETZKY, M. 2005. Smooth feature lines on surface meshes. In *SGP '05: Proceedings of the third Eurographics symposium on Geometry processing*, Eurographics Association, Aire-la-Ville, Switzerland, Switzerland, 85.
- HORMANN, K., LÉVY, B., AND SHEFFER, A. 2007. Mesh parameterization: theory and practice. In *SIGGRAPH '07: ACM SIGGRAPH 2007 courses*, 1.
- HUANG, J., ZHANG, M., MA, J., LIU, X., KOBBELT, L., AND BAO, H. 2008. Spectral quadrangulation with orientation and alignment control. *ACM Trans. Graph.* 27, 5, 1–9.
- KÄLBÉRER, F., NIESER, M., AND POLTHIER, K. 2007. Quadcover - surface parameterization using branched coverings. *Computer Graphics Forum* 26, 3 (Sept.), 375–384.
- KHAREVYCH, L., SPRINGBORN, B., AND SCHRÖDER, P. 2006. Discrete conformal mappings via circle patterns. *ACM Trans. Graph.* 25, 2, 412–438.
- LAI, Y.-K., KOBBELT, L., AND HU, S.-M. 2008. An incremental approach to feature aligned quad dominant remeshing. In *SPM '08: Proceedings of the 2008 ACM symposium on Solid and physical modeling*, 137–145.
- MARINOV, M., AND KOBBELT, L. 2004. Direct anisotropic quad-dominant remeshing. In *PG '04: Proceedings of the Computer Graphics and Applications, 12th Pacific Conference*, IEEE Computer Society, Washington, DC, USA, 207–216.
- RAY, N., LI, W. C., LÉVY, B., SHEFFER, A., AND ALLIEZ, P. 2006. Periodic global parameterization. *ACM Trans. Graph.* 25, 4, 1460–1485.
- RAY, N., VALLET, B., ALONSO, L., AND LÉVY, B. 2008. Geometry aware direction field design. Tech. rep., INRIA - ALICE Project Team. Accepted pending revisions.
- RAY, N., VALLET, B., LI, W. C., AND LÉVY, B. 2008. N-symmetry direction field design. *ACM Trans. Graph.* 27, 2, 1–13.
- SPRINGBORN, B., SCHRÖDER, P., AND PINKALL, U. 2008. Conformal equivalence of triangle meshes. In *SIGGRAPH '08: ACM SIGGRAPH 2008 papers*, 1–11.
- TONG, Y., ALLIEZ, P., COHEN-STEINER, D., AND DESBRUN, M. 2006. Designing quadrangulations with discrete harmonic forms. In *Proc. SGP*, Eurographics Association, 201–210.
- ZHANG, E., MISCHAIKOW, K., AND TURK, G. 2006. Vector field design on surfaces. *ACM Trans. Graph.* 25, 4, 1294–1326.

ACM Transactions on Graphics, Vol. 28, No. 3, Article 77, Publication date: August 2009.